

Universidade de Brasília
Departamento de Ciência da Computação
Teleinformática e Redes 02

Relatório da Primeira Entrega — Trabalho Final

Cliente de Streaming Adaptativo com Algoritmo ABR Rate-Based
Relação Client-Server

Grupo: 15

Nome: Luiz Fernando Candido Soaris

Matrícula: 190092025

Nome: Isabela Soares Furlan

Matrícula: 231013636

Nome: Daniel Lincoln Lopes de Souza

Matrícula: 202042829

Professor:

Prof. **Jacir Luiz Bordim**

Brasília, Maio de 2026

Introdução

Este trabalho implementa um cliente de *streaming* de vídeo adaptativo utilizando o algoritmo ABR *Rate-Based* puro, desenvolvido para a disciplina Teleinformática e Redes 02. O cliente realiza o download de 10 segmentos de vídeo a partir de um servidor HTTP, selecionando automaticamente a qualidade com base na vazão medida da rede. O sistema foi estruturado em quatro módulos Python (`main.py`, `buffer_manager.py`, `metrics_logger.py`, `metrics_plotter.py`), registra 14 métricas por segmento em CSV e gera gráficos de desempenho para análise.

Metodologia

Arquitetura do Sistema

O projeto foi dividido em quatro módulos com responsabilidades bem definidas:

Módulo	Responsabilidade
<code>main.py</code>	Coordena o loop de download dos 10 segmentos; implementa o algoritmo Rate-Based com janela móvel de 5 amostras; calcula jitter e EWMA
<code>buffer_manager.py</code>	Simula o buffer do player: incrementa a cada segmento recebido e decai com o tempo real; detecta eventos de <i>rebuffering</i>
<code>metrics_logger.py</code>	Registra os 14 campos de métrica por segmento e exporta para CSV
<code>metrics_plotter.py</code>	Gera os quatro gráficos de desempenho via matplotlib

Algoritmo Rate-Based

O algoritmo funciona da seguinte forma: a cada segmento, calcula-se a média das últimas 5 vazões medidas e aplica um fator de segurança de 0,85, obtendo a banda segura $v_{safe} = \bar{v} \times 0,85$. Em seguida, seleciona-se a maior representação cujo bitrate não exceda v_{safe} . O fator 0,85 cria uma margem de 15% para absorver flutuações bruscas de rede. A janela de 5 amostras equilibra estabilidade e reatividade.

```
1 def select_quality(representations, avg, safety_factor=0.85):
2     safe = avg * safety_factor
3     selected = representations[0] # fallback: menor qualidade
4     for rep in representations:
5         if rep["bitrate_kbps"] > safe:
6             break
7         selected = rep
8     return selected
```

Listing 1: Seleção de qualidade Rate-Based (`main.py`)

Gerenciamento de Buffer

O BufferManager simula o buffer do player: a cada segmento recebido, `buffer_level += 2s` (duração do segmento); continuamente, `buffer_level -= tempo_decorrido`. Quando o buffer chega a zero, um evento de *rebuffering* é registrado e o player aguarda 1 segundo antes de continuar.

```
1 def update_decay(self):
2     elapsed = time.time() - self.last_update_time
3     self.last_update_time = time.time()
4     self.buffer_level = max(0, self.buffer_level - elapsed)
5     if self.buffer_level == 0 and not self.is_rebuffering:
6         self.is_rebuffering = True
7         self.rebuffering_events += 1
8
9 def add_segment(self):
10     self.update_decay()
11     self.buffer_level += self.segment_duration
```

Listing 2: Decaimento e incremento do buffer (buffer_manager.py)

Métricas Registradas

São registrados 14 campos por segmento no CSV: `segment`, `timestamp`, `server_id`, `quality`, `bitrate_kbps`, `vazao_kbps`, `download_time_s`, `jitter_network_ms`, `jitter_ewma_ms`, `buffer_level_s`, `buffer_can_play`, `rebuffer_event`, `stall_duration_s` e `failover_total`. O jitter de rede é calculado como $|t_{chegada} - t_{esperado}|$ e o jitter EWMA suaviza esse valor com $\alpha = 0,2$.

Resultados e Discussão

Desempenho por Segmento

Table 1: Métricas por segmento — CSV `metrics_20260520_210956.csv`

Seg.	Qualidade	Bitrate (kbps)	Vazão (kbps)	Buffer (s)	Rebuff./Stall
1	240p	200	1167,02	1,00	Sim / 1,00s
2	480p	700	1140,48	2,38	Não
3	480p	700	1230,90	3,81	Não
4	480p	700	1253,08	5,26	Não
5	480p	700	1243,95	6,69	Não
6	480p	700	1170,84	8,10	Não
7	480p	700	1287,70	9,55	Não
8	480p	700	1231,63	10,98	Não
9	480p	700	1175,70	12,39	Não
10	480p	700	1211,21	13,89	Não
Média			1211,25	7,41	1 evento / 1,00s

Qualidade Seleccionada

O segmento 1 foi baixado em 240p pois não havia histórico de vazão — a média retorna 0 e o algoritmo recorre ao *fallback* (menor qualidade). A partir do segmento 2, com vazão medida de 1.167 kbps, $v_{safe} \approx 992$ kbps, e a qualidade escolhida foi 480p (700 kbps). O 720p (1.500 kbps) ficou fora por exigir vazão ≥ 1.765 kbps, que nunca foi atingida. A qualidade permaneceu estável em 480p até o segmento 10.

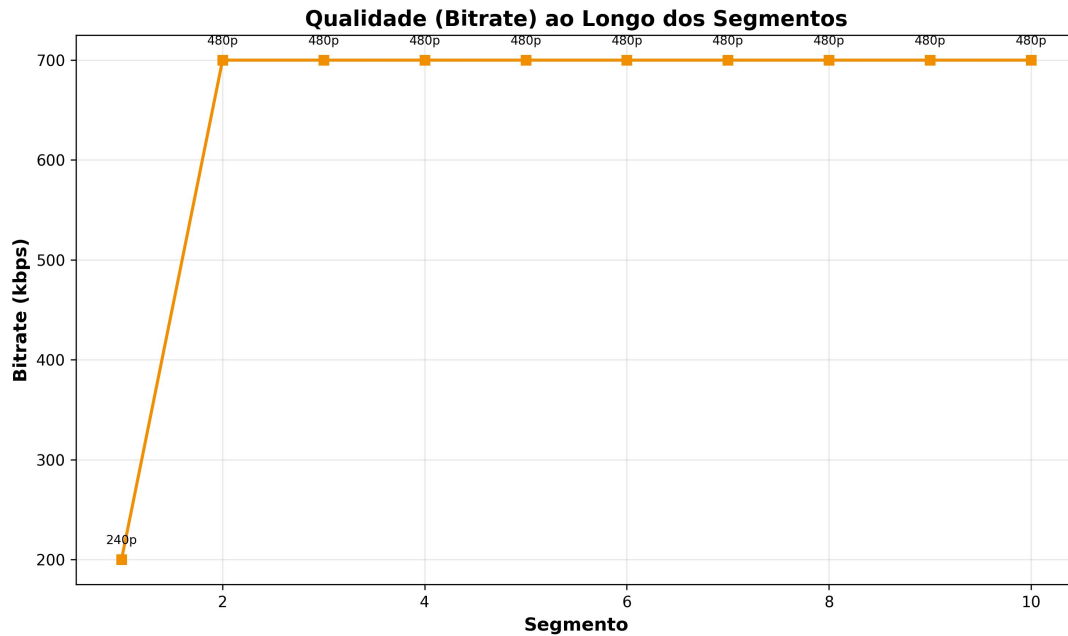


Figure 1: Qualidade (bitrate em kbps) selecionada por segmento. Estabilização imediata em 480p a partir do segmento 2.

Vazão

A vazão oscilou entre 1.140 e 1.288 kbps com média de 1.211 kbps, o que é típico de redes IP reais. A janela móvel de 5 amostras suaviza parte dessas variações, mantendo a seleção de qualidade estável mesmo com flutuações de curto prazo.

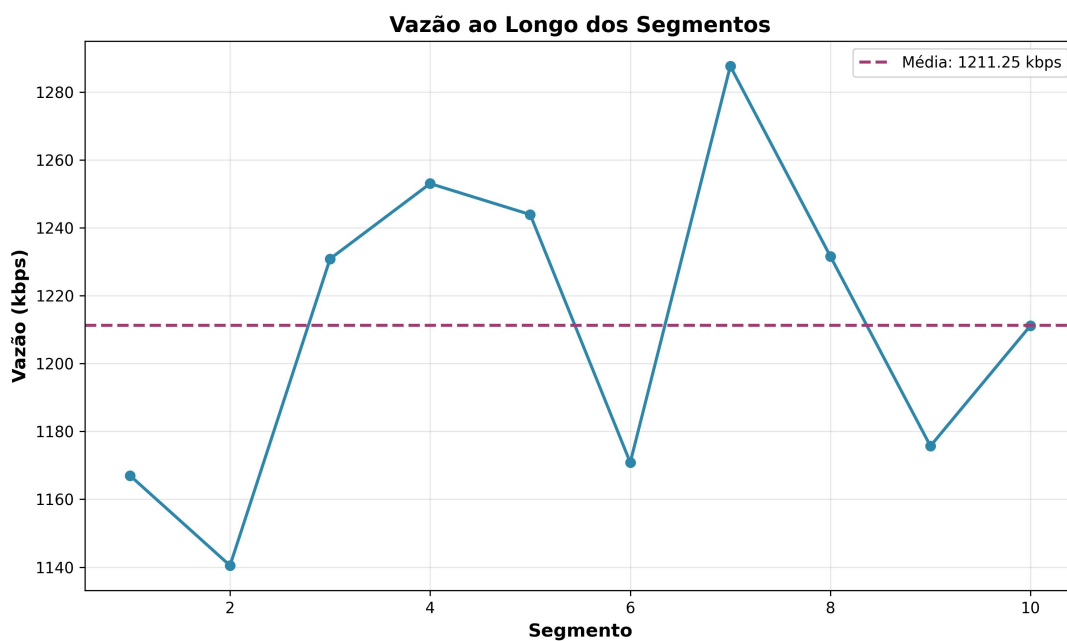


Figure 2: Vazão medida por segmento (kbps). Linha tracejada indica a média geral de 1.211,25 kbps.

Nível do Buffer

O buffer cresceu linearmente de 1,00s a 13,89s porque a vazão (≈ 1.211 kbps) é bem maior que o bitrate selecionado (700 kbps). Cada segmento de 2s é baixado em $\approx 0,57$ s, gerando um ganho líquido de $\approx 1,43$ s de buffer por ciclo. O buffer ficou sempre acima do mínimo de 2s a partir do segmento 2, garantindo reprodução contínua.

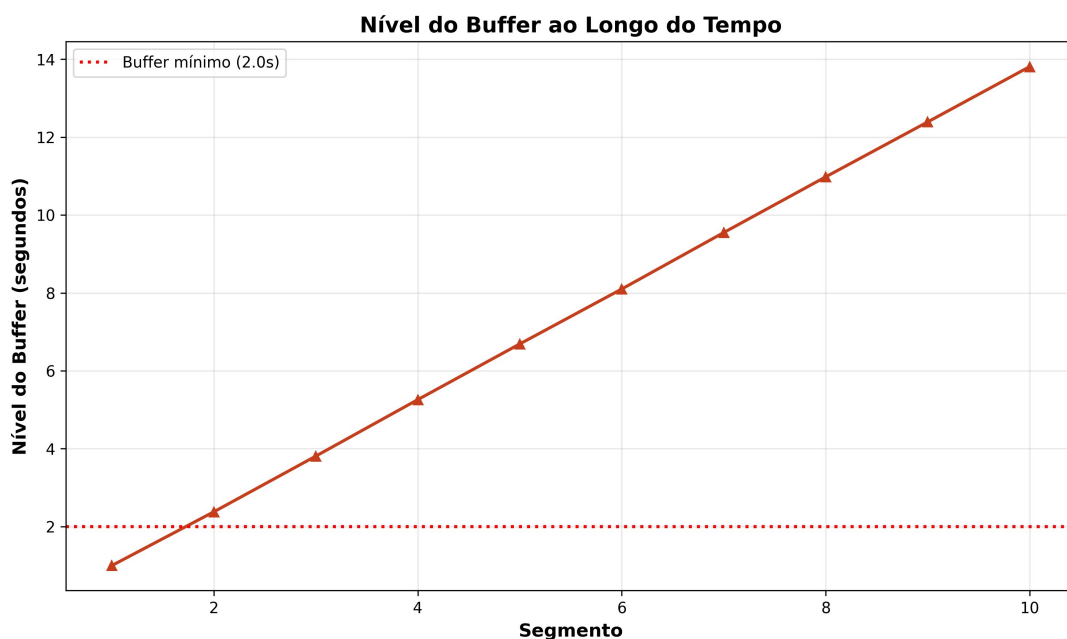


Figure 3: Nível do buffer ao longo dos segmentos. Crescimento linear bem acima do limiar mínimo de 2s.

Gráfico Combinado

A Figura 4 reúne vazão e qualidade em um único painel, facilitando a visualização da correlação entre as métricas. Note que a qualidade é estritamente estável mesmo com oscilações de vazão, evidenciando a eficácia da janela móvel e do fator de segurança.

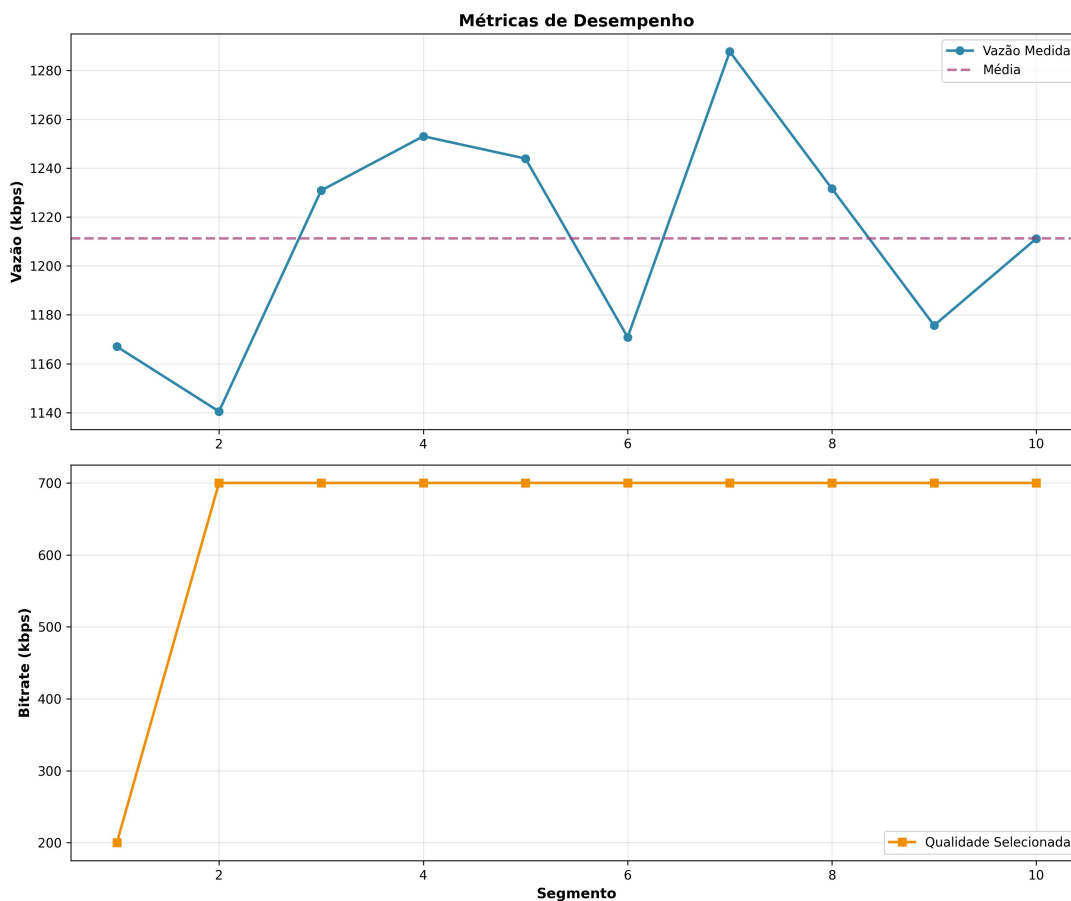


Figure 4: Métricas combinadas: vazão (acima) e bitrate selecionado (abaixo) por segmento.

Conclusão

O cliente ABR Rate-Based implementado funcionou conforme o esperado: convergiu rapidamente para 480p, manteve o buffer crescendo de forma saudável e registrou apenas 1 evento de *rebuffering* no início da sessão. Todas as 14 métricas foram corretamente gravadas no CSV e os quatro gráficos gerados refletem o comportamento observado nos logs do terminal. As limitações do Rate-Based puro ficam evidentes: a qualidade ficou aquém do potencial real da rede (720p seria viável se o fator de segurança fosse mais agressivo) e o algoritmo não antecipa tendências, reagindo apenas após variações acontecerem. Essas deficiências motivam a implementação de políticas híbridas nas próximas entregas.